

DATA CLEANING, LOADING & EXPLORATORY ANALYSIS

Import libraries and load data.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10,6)
plt.rcParams['font.size'] = 12

df = pd.read_csv('Titanic-Dataset.csv')

print('Dataset Loaded!')
print(f'Shape of the dataset: {df.shape}')
```

Dataset Loaded!
Shape of the dataset: (891, 12)

View the first 5 rows.

df.head()

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S

Next steps: [Generate code with df](#) [New interactive sheet](#)

Get basic info about the dataset.

```
print('=== Dataset Info ===')
df.info()

=== Dataset Info ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Descriptive statistics for all numerical columns.

```
print('=== Descriptive Statistics ===')
df.describe().round(2)
```

=== Descriptive Statistics ===

1 to 8 of 8 entries Filter ?

index	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.0	891.0	891.0	714.0	891.0	891.0	891.0
mean	446.0	0.38	2.31	29.7	0.52	0.38	32.2
std	257.35	0.49	0.84	14.53	1.1	0.81	49.69
min	1.0	0.0	1.0	0.42	0.0	0.0	0.0
25%	223.5	0.0	2.0	20.12	0.0	0.0	7.91
50%	446.0	0.0	3.0	28.0	0.0	0.0	14.45
75%	668.5	1.0	3.0	38.0	1.0	0.0	31.0
max	891.0	1.0	3.0	80.0	8.0	6.0	512.33

Show 25 per page

Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

Check value counts for categorical columns.

```
print('=== Survival count ===')
print(df['Survived'].value_counts())
print(df['Survived'].value_counts(normalize=True).round(3)*100)

print('\n=== Passenger Class count ===')
print(df['Pclass'].value_counts().sort_index())

print('\n=== Gender count ===')
print(df['Sex'].value_counts())

print('\n=== Embarked count ===')
print(df['Embarked'].value_counts())
```

```
=== Survival count ===
Survived
0    549
1    342
Name: count, dtype: int64

Survived
0    61.6
1    38.4
Name: proportion, dtype: float64

=== Passenger Class count ===
Pclass
1     216
2     184
3     491
Name: count, dtype: int64

=== Gender count ===
Sex
male     577
female   314
Name: count, dtype: int64

=== Embarked count ===
Embarked
S     646
C     168
Q       77
Name: count, dtype: int64
```

Identify all missing values.

```
missing = df.isnull().sum()
missing_pct = (df.isnull().sum() / len(df)*100).round(2)

missing_df = pd.DataFrame({'Missing': missing, 'Missing %': missing_pct})
missing_df.sort_values(by='Missing', ascending=False)
```

```
print('=== Missing Values Summary ===')
print(missing_df[missing_df['Missing']>0])
```

```
=== Missing Values Summary ===
      Missing  Missing %
Age         177      19.87
Cabin        687      77.10
Embarked      2       0.22
```

Handle missing values - 3 different strategies.

```
df.drop(columns = ['Cabin'], inplace=True, errors='ignore')
print('Cabin column dropped.')

age_medium = df['Age'].median()
print(f'Age column filled with {age_medium}.')
df['Age'].fillna(age_medium, inplace=True)

embarked_mode = df['Embarked'].mode()[0]
print(f'Embarked column filled with {embarked_mode}.')
df['Embarked'].fillna(embarked_mode, inplace=True)

print('\nMissing values after cleaning:')
print(df.isnull().sum()[df.isnull().sum()>0])
print('No missing values!' if df.isnull().sum().sum()==0 else 'Missing values still exist!')
```

```
Cabin column dropped.
Age column filled with 28.0.
Embarked column filled with S.
```

```
Missing values after cleaning:
Series([], dtype: int64)
No missing values!
```

Check for duplicate rows.

```
duplicates = df.duplicated().sum()
print(f'Number of duplicate rows: {duplicates}')

df.drop_duplicates(inplace=True)
print(f'Number of duplicate rows after dropping: {df.duplicated().sum()} (0)' if duplicates>0 else 'No duplicate rows!')
```

```
Number of duplicate rows: 0
No duplicate rows!
```

Create a new useful columns (Feature Engineering).

```
bins = [0, 12, 18, 35, 60, 100]
labels = ['Child', 'Teenager', 'Young Adult', 'Adult', 'Senior']
df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels)

df['FamilySize'] = df['SibSp'] + df['Parch'] + 1

df['IsAlone'] = (df['FamilySize'] == 1).astype(int)

print('New columns added: AgeGroup, FamilySize, IsAlone')
print(df[['Age', 'Age_Group', 'SibSp', 'Parch', 'FamilySize', 'IsAlone']].head(10))
```

```
New columns added: AgeGroup, FamilySize, IsAlone
   Age  Age_Group  SibSp  Parch  FamilySize  IsAlone
0  22.0  Young Adult     1     0           2         0
1  38.0     Adult     1     0           2         0
2  26.0  Young Adult     0     0           1         1
3  35.0  Young Adult     1     0           2         0
4  35.0  Young Adult     0     0           1         1
5  28.0  Young Adult     0     0           1         1
6  54.0     Adult     0     0           1         1
7   2.0     Child     3     1           5         0
8  27.0  Young Adult     0     2           3         0
9  14.0  Teenager     1     0           2         0
```

Survival rate by different groups.

```

print('=== Survival rate by gender ===')
print(df.groupby('Sex')['Survived'].mean().round(3)*100)

print('\n=== Survival rate by passenger class ===')
print(df.groupby('Pclass')['Survived'].mean().round(3)*100)

print('\n=== Survival rate by age group ===')
print(df.groupby('Age_Group')['Survived'].mean().round(3)*100)

print('\n=== Average fare by class ===')
print(df.groupby('Pclass')['Fare'].mean().round(2))

print('\n=== Correlation Matrix ===')
print(df[['Survived', 'Pclass', 'Age', 'Fare',
          'FamilySize']].corr().round(3))

```

```

=== Survival rate by gender ===
Sex
female    74.2
male      18.9
Name: Survived, dtype: float64

=== Survival rate by passenger class ===
Pclass
1     63.0
2     47.3
3     24.2
Name: Survived, dtype: float64

=== Survival rate by age group ===
Age_Group
Child          58.0
Teenager       42.9
Young Adult    35.3
Adult          40.0
Senior         22.7
Name: Survived, dtype: float64

=== Average fare by class ===
Pclass
1     84.15
2     20.66
3     13.68
Name: Fare, dtype: float64

=== Correlation Matrix ===
           Survived  Pclass   Age   Fare  FamilySize
Survived      1.000  -0.338 -0.065  0.257      0.017
Pclass       -0.338   1.000 -0.340 -0.549      0.066
Age          -0.065  -0.340   1.000  0.097     -0.246
Fare          0.257  -0.549  0.097   1.000      0.217
FamilySize    0.017   0.066 -0.246  0.217      1.000

```